Music Curator - Sprint Master Task Grab List

Repo: https://github.com/tolmarr/Music-Curator
Schema: `sql/schema.sql`
ERD: `project_kick_off_files/ERD_CHEN_NOTATION.png`

Scope: New features and design changes only. Existing `Favourites`, `AlbumSongs`, `Performs`, `ArtistAlbums`, and completed relationship-table work do **not** need to be rebuilt or re-documented unless a new feature directly changes them.

1. How claiming works

1. Put your name beside **one task** below. Coordinate if two tasks touch the same table.
2. If you modify a relationship table, also verify **both endpoint entity tables** and their keys/foreign keys. Reuse existing tables; don't recreate them or redo finished work.
3. For each task, record **what changed, why the design makes sense, any constraints/limitations, and your branch/commit**. Basic schema checks are enough for now; no application-function testing required.
4. Coordinate schema changes so `sql/schema.sql` can still build the database from scratch. The final ERD must match the final implementation.

2. Task board

P0: Required new features | **P1:** Final integration | **P2:** Future material (not yet taught)

ID	Priority	New task	Claimed by	Done
T01	P0	Add review text to `Rates`	Aga	☐
T02	P0	Add Certified Curator status and password hash to `Users`	Chris	☐
T03	P0	Fix `Lists` using `SongLists` and `AlbumLists` subtypes	Aga	☐

ID	Priority	New task	Claimed by	Done
T04	P0	Document and implement new review/list constraints	Cooper	☐
T05	P0	Define trending albums from ratings and curation	Cooper	☐
T06	P0	Define certified-first review ordering for Reviews Wall data	Tola	☐
T07	P1 (last)	Update ERD and integrate/document new changes	Chris	☐

3. Task implementation cards

T01 — Add written album reviews

Tables: `Rates`; check its existing endpoints `Users` and `Albums`.
Depends on: None (coordinate with T02, T04, T06).
Lesson: ER relationship attributes; SQL DDL.

- Add a `review TEXT` attribute to the already implemented `Rates` relationship.
- Allow letters, numbers, sentences, and paragraphs. Decide whether a rating may have no written review (`NULL`).
- Preserve the existing 1–5 rating rule and one rating per `(username, album_id)`; do not rebuild them.
- Document the updated attribute and whether blank/whitespace-only reviews count as a review.

T02 — Add Certified Curators and necessary user attributes

Tables: `Users`.
Depends on: None (coordinate with T01 and T06).
Lesson: Attributes, identifiers, and conceptual minimality.

- Add `is_certified_curator BOOLEAN NOT NULL DEFAULT FALSE` (or equivalent).
- Add `password_hash VARCHAR(255)` for stored password hashes, **not plaintext passwords**. Authentication/UI is out of scope.
- Keep `username` and `display_name` and avoid unnecessary new profile attributes.
- Document that certification must be assigned by an authorized process, not freely chosen by users. That permission check is not automatically enforced by a Boolean field.

T03 — Correct the `Lists` inheritance design

Tables: `Lists`; new `SongLists` and `AlbumLists`; adjust references from existing `ListSongs`/`ListAlbums` only as needed.
Depends on: Coordinate with T05.
Lesson 09: *Weak Entity Sets & Inheritance*.

- Make `SongLists` and `AlbumLists` actual subtypes of `Lists` (not simply names for the existing relationship tables).
- Each subtype's `list_id` is both **PK** and **FK** to `Lists(list_id)`.
- Make the existing song/album membership relationships point to their corresponding subtype.
- Decide how to avoid a conflicting `Lists.list_type` value or a list appearing in both subtypes; document any rule that requires enforcement beyond the FKs.
- Update the inheritance explanation for the final ERD. Do not redo completed list ordering functionality.

T04 — Express new rules as relational algebra constraints

Rates

Let W^+ be the set of whitespace strings of length ≥ 1
Let B (blank strings) = { s ∈ Strings | |s|=0 ∨ s ∈ W+ }
$\sigma_{review \in B}$ (Rates) = ∅

SongLists

Let S be the set of list_id values in SongLists
Let L be the set of list_id values in Lists
Every ID in SongLists must also be in Lists
π_{list_id}(SongLists) ⊆ π_{list_id}(Lists)

AlbumLists

Let S be the set of list_id values in AlbumLists
Let L be the set of list_id values in Lists
Every ID in AlbumLists must also be in Lists
$\pi_{list_id}(AlbumLists) \subseteq \pi_{list_id}(Lists)$

Tables: New `Rates.review` rule and the revised list subtypes.
Depends on: T01, T03.
Lesson 08: *Constraints in Relational Algebra*.

- Write **at least two** constraints in English and relational algebra, for example: non-blank written reviews (when present) and every `SongLists.list_id` referring to an existing `Lists.list_id`.
- Demonstrate both an **empty-result constraint** (`invalid_rows = ∅`) and a **subset/referential-integrity constraint** (`child_ids ⊆ parent_ids`).
- Note which constraints SQL DDL enforces and which require application logic or another mechanism.
- Keep evidence short; this task is about the **new rules**, not documenting every old constraint again.

T05 — Define how trending albums are calculated

Tables used: `Albums`, `Rates`, `Lists`, `ListAlbums` (and T03's subtype if used).
Depends on: T03 for final list design.
Lesson 10: *Improving Conceptual Designs* — minimality and derived data.

- Define `trending(album)` as the count of **distinct users** who rated the album or added it to an album list.
- A user contributes only **one** count per album, even if they rated it and curated it or placed it in multiple lists.
- Document a SQL query or view design to derive this count from existing relations.
- Do **not** add a stored `trending` count to `Albums` unless the group later has a strong reason: it is derivable and would introduce redundancy.
- The Reviews Wall display and page layout are **frontend**, not part of this task.

T06 — Design certified-first review retrieval

Tables used: `Rates`, `Users`, `Albums`.
Depends on: T01, T02 (and coordinates with T05).
Lesson: Relational queries and conceptual design for new requirements.

- Specify how the database retrieves written reviews for a chosen album, together with each reviewer's certification status.
- Return Certified Curator reviews **before** other reviews; choose and document a secondary order for ties.

- Keep `is_certified_curator` on `Users`, not duplicated on individual reviews; joins supply the status.
- Do not implement cards, styling, navigation, or the visual Reviews Wall.

T07 — Update ERD and documentation after implementation

Files: ERD, `sql/schema.sql`, `README.md`.
Depends on: T01–T06; do **last**.
Lesson 10: *Improving Conceptual Designs* — completeness, correctness, minimality, expressiveness, readability, extensibility.

- Update the Chen-notation ERD with `Rates.review`, new `Users` attributes, and any adopted `Lists` inheritance.
- Show correct identifiers, multiplicities, and subtype relationships. Ensure the diagram matches the merged schema.
- Explain briefly how the new design improves at least **three** qualities from the lesson, especially minimality (derived trending count), correctness (proper subtype representation), and completeness (reviews/curators).
- Verify `sql/schema.sql` can build the full database from scratch and update the README with only the new requirements and any remaining enforcement limits.

4. Task documentation template

Task ID / Owner:
Tables/files changed:
Changes made:
Why this design? (Course concept):
Keys, relationships, and new constraints:
What SQL enforces / what it doesn't:
Basic schema check (if relevant):
Branch or commit:
Outstanding coordination:

5. Later lessons — not part of this sprint

ERD-derived functional dependencies, minimal FD bases, BCNF, 3NF, and 4NF will be revisited **after** those lessons are taught. Do not add tasks for them yet.

Design Concept for Review Wall / Landing Page

